ternational Conference on Learning Representations (ICLR), arXiv preprint arXiv:1801.09344.

Raghunathan, A., Steinhardt, J., and Liang, P. S. (2018b). Semidefinite relaxations for certifying robustness to adversarial examples. In *Advances in Neural Information Processing Systems*, pages 10900–10910.

Salman, H., Yang, G., Zhang, H., Hsieh, C.-J., and Zhang, P. (2019). A convex relaxation barrier to tight robust verification of neural networks. *arXiv preprint arXiv:1902.08722.*

Singh, G., Gehr, T., Mirman, M., Püschel, M., and Vechev, M. (2018). Fast and effective robustness certification. In *Advances in Neural Information Processing Systems*, pages 10825–10836.

Singh, G., Gehr, T., Püschel, M., and Vechev, M. (2019a). An abstract domain for certifying neural networks. *Proceedings of the ACM on Programming Languages*, 3(POPL):41.

Singh, G., Gehr, T., Püschel, M., and Vechev, M. (2019b). Robustness certification with refinement. *ICLR.*

Sinha, A., Namkoong, H., and Duchi, J. (2018). Certifying some distributional robustness with principled adversarial training. In *ICLR.*

Strauss, T., Hanselmann, M., Junginger, A., and Ulmer, H. (2017). Ensemble methods as a defense to adversarial perturbations against deep neural networks. *arXiv preprint arXiv:1709.03423.*

Tjeng, V., Xiao, K., and Tedrake, R. (2019). Evaluating robustness of neural networks with mixed integer programming. *ICLR.*

Uesato, J., O'Donoghue, B., Oord, A. v. d., and Kohli, P. (2018). Adversarial risk and the dangers of evaluating against weak attacks. *International Conference on Machine Learning (ICML).*

Wang, S., Chen, Y., Abdou, A., and Jana, S. (2018a). Mixtrain: Scalable training of formally robust neural networks. *arXiv preprint arXiv:1811.02625.*

Wang, S., Pei, K., Whitehouse, J., Yang, J., and Jana, S. (2018b). Efficient formal safety analysis of neural networks. In *Advances in Neural Information Processing Systems*, pages 6369–6379.

Weng, T.-W., Zhang, H., Chen, H., Song, Z., Hsieh, C.-J., Boning, D., Dhillon, I. S., and Daniel, L. (2018). Towards fast computation of certified robustness for ReLU networks. In *International Conference on Machine Learning.*

Wong, E. and Kolter, Z. (2018). Provable defenses against adversarial examples via the convex outer adversarial polytope. In *International Conference on Machine Learning (ICML)*, pages 5283–5292.

Wong, E., Schmidt, F., Metzen, J. H., and Kolter, J. Z. (2018). Scaling provable adversarial defenses. *Advances in Neural Information Processing Systems (NIPS).*

Xiao, K. Y., Tjeng, V., Shafiullah, N. M., and Madry, A. (2019). Training for faster adversarial robustness verification via inducing relu stability. *ICLR.*

Zhang, H., Weng, T.-W., Chen, P.-Y., Hsieh, C.-J., and Daniel, L. (2018). Efficient neural network robustness certification with general activation functions. In *Advances in Neural Information Processing Systems (NIPS).*

Zhang, H., Zhang, P., and Hsieh, C.-J. (2019). Recurjac: An efficient recursive algorithm for bounding jacobian matrix of neural networks and its applications. *AAAI Conference on Artificial Intelligence.*

A The Coordinate Descent Algorithm for RobBoost

Algorithm 1 One-step Coordinate Decent Update

Input: $N \times (C-1)$ vectors $\boldsymbol{\omega}^{j,k} \in \mathbb{R}^d$, $\boldsymbol{\nu}^{j,k} \in \mathbb{R}^d$ and $N \times (C-1)$ scalars $\gamma^{j,k}$ and $\mu^{j,k}$ which are formed based (15), $j \in [C-1], k \in [N]$, current weight vector $\boldsymbol{\alpha}$ and the coordinate t to be updated

Output: an updated weight α_t for coordinate t

for $(j,k) \in [C-1] \times [N]$ **do**
  Solve $r_1^{j,k}, \cdots, r_d^{j,k}$, where $\alpha_t \boldsymbol{\omega}_l^{j,k} + \boldsymbol{\nu}_l^{j,k} = 0, \ \ l \in [d]$
  Sort $r_1^{j,k}, \cdots, r_d^{j,k}$ in ascending order $\pi^{j,k}$
  Compute $J_+^{j,k}$, $J_-^{j,k}$ according to (16) and (17)
  Compute $a_0^{j,k}$, $b_0^{j,k}$ according to (18) and (19)
  Initial $N \times (C-1)$ index counters $m^{j,k} \leftarrow 0$
Merge sort all $N \times (C-1)$ lists $r^{j,k}$ into one list r with $N \times (C-1) \times d$ elements in ascending order π
Maintain two mappings $p(i) \to (j,k)$, $q(i) \to l$ where r_i comes from the element $r_l^{j,k}$ during merge
Initial list $h^{j,k}$ where $h^{j,k} \leftarrow \frac{-b_0^{j,k}}{a_0^{j,k}}$ if $\frac{-b_0^{j,k}}{a_0^{j,k}} > 0$; ∞ otherwise {stores all the positive 0-crossing points}
$L_b \leftarrow \sum_{k \in [N]} \sum_{j \in [C-1]} \max(b_0^{j,k}, 0)$, $\alpha_{\text{best}} \leftarrow 0$ {Initial best loss, best α_t at point $\alpha_t = 0$}
$i \leftarrow 1$
while $i \leq N \times (C-1) \times d$ **do**
  if $r_{\pi(i)} \leq 0$ or $r_{\pi(i)} \geq 1$ **then**
    $(j,k) \leftarrow p(i)$, $M \leftarrow m^{j,k} + 1$, $m^{j,k} \leftarrow m^{j,k} + 1$
    $a_M^{j,k} \leftarrow a_{M-1}^{j,k}$ {Skip all $r_{\pi(i)}$ out of the $[0,1]$ range}
    $b_M^{j,k} \leftarrow b_{M-1}^{j,k}$
    continue
  $(j',k') \leftarrow \arg\min_{j,k} h^{j,k}$
  if $r_{\pi(i)} > h^{j',k'}$ and $h^{j',k'} > 0$ **then**
    $x \leftarrow h^{j',k'}$ {Evaluate at a 0-crossing point for term (j',k')}
    $h^{j',k'} \leftarrow \infty$ {Note that we do not increment counter i in this case}
  else
    $(j,k) \leftarrow p(i)$, $M \leftarrow m^{j,k} + 1$, $m^{j,k} \leftarrow m^{j,k} + 1$
    Update $a_M^{j,k}$, $b_M^{j,k}$ recursively using $a_{M-1}^{j,k}, b_{M-1}^{j,k}$ based on (20) and (21)
    if $\frac{-b_M^{j,k}}{a_M^{j,k}} > r_{\pi(i)}$ **then**
      $h^{j,k} \leftarrow \frac{-b_M^{j,k}}{a_M^{j,k}}$ {Update the 0-crossing point we might encounter}
    else
      $h^{j,k} \leftarrow \infty$ {This piece does not cross 0 when we scan forward}
    $x \leftarrow r_{\pi(i)}$
    $i \leftarrow i + 1$
  Compute $g^{j,k}(x) = a_{m^{j,k}}^{j,k} x + b_{m^{j,k}}^{j,k}$, for all $(j,k) \in [C-1] \times [N]$
  Compute Loss at current point $L = \sum_{k \in [N]} \sum_{j \in [C-1]} \max(g^{j,k}(x), 0)$
  if $L < L_b$ **then**
    $L_b \leftarrow L, \ \ \alpha_{\text{best}} \leftarrow x$ {Update best loss}
return α_{best}

Here we present our full algorithm to optimally update one coordinate in coordinate descent. First, we rewrite our objective function:

$$g(\alpha_t) = \sum_{k \in [N]} \sum_{j \in [C-1]} \max(\|\alpha_t \boldsymbol{\omega}^{j,k} + \boldsymbol{\nu}^{j,k}\|_1 + \gamma^{j,k} \alpha_t + \mu^{j,k} + 1, 0) \tag{24}$$

$$:= \sum_{k \in [N]} \sum_{j \in [C-1]} \max(g^{j,k}(\alpha_t), 0) \tag{25}$$

There are $N \times (C-1)$ terms in summation (25), and we need to maintain "effective slope" $a^{j,k}$ and "effective

intercept" $b^{j,k}$ for each term (j,k) using the technique presented in the main text. Thus most variables have superscript (j,k). Note that to deal with the max function in hinge loss, for each term we need to dynamically maintain the possible zero crossing point, which is stored in $h^{j,k}$ (line 9). If we reached a zero-crossing point before any other $r_{\pi(i)}$, we need to evaluate the function value at this zero-crossing point first (line 19). Additionally, once the effective slope $a_M^{j,k}$ and effective intercept $b_M^{j,k}$ change due to a sign change in absolute value terms, we need to update the zero-crossing point $h^{j,k}$ (line 25). In each iteration of coordinate descent, we choose a coordinate t, obtain (14) using Algorithm 1 and set $\alpha_t \leftarrow \alpha_t^*$. Initially, we can set $\alpha_t = \frac{1}{T}$, then we can run several epochs of coordinate descent; in each epoch we optimize over all α_t once, either in a cyclic order or a random order. Although coordinate descent cannot always find the global optimal solution, we found that it usually reduces the loss function sufficiently and rapidly; 2 or 3 epochs are sufficient for all our experiments.

B Model Details

We implement all our models using PyTorch. Using model structures detailed in Table 4, we obtain 12 MNIST models and 11 CIFAR-10 models (without the last largest ResNet due to out of memory), robustly trained using the method proposed in Wong et al. (2018). For MNIST and CIFAR-10 we use the same model structure; only the input image shape differs.

For experiments on ensemble of robustly trained models, we use all available models. For experiments on feature subsample, we use model K for both MNIST and CIFAR-10 datasets. For experiments on the ensemble of graident boosted models, we use model A,C,E,G,K for MNIST and B,G,I,J,K for CIFAR-10 dataset.

Table 4: Models structures. All layers are followed by ReLU activations. The last fully connected layer with a fixed 10-dimensional output is omitted. "Conv $kw \times h + s$" corresponds to a 2D convolutional layer with k filters of size $w \times h$ using a stride of s in both dimensions. $[\cdot]$ stands for a Resnet block.

Name	Model details
A	Conv 4 4×4+2, Conv 8 4×4+2, FC 128
B	Conv 8 4×4+2, Conv 16 4×4+2, FC 256
C	Conv 4 3×3+1, Conv 8 3×3+1, Conv 8 4×4+4, FC 64
D	Conv 8 3×3+1, Conv 16 3×3+1, Conv 16 4×4+4, FC 128
E	Conv 4 5×5+1, Conv 8 5×5+1, Conv 8 5×5+4, FC 64
F	Conv 8 5×5+1, Conv 16 5×5+1, Conv 16 5×5+4, FC 128
G	Conv 4 3×3+1, Conv 4 4×4+2, Conv 8 3×3+1, Conv 8 4×4+2, FC 256, FC 256
H	Conv 4 3×3+1, Conv 4 4×4+2, Conv 8 3×3+1, Conv 8 4×4+2, FC 512, FC 512
I	Conv 8 3×3+1, Conv 8 4×4+2, Conv 16 3×3+1, Conv 16 4×4+2, FC 256, FC 256
J	Conv 8 3×3+1, Conv 8 4×4+2, Conv 16 3×3+1, Conv 16 4×4+2, FC 512, FC 512
K	Conv 4 3×3+1, [Conv 8 4×4+2, Conv 8 3×3+1 Conv 8 3×3+1], [Conv 8 4×4+2, Conv 8 3×3+1 Conv 8 3×3+1], FC 128
L	Conv 8 3×3+1, [Conv 16 4×4+2, Conv16 3×3+1 Conv 16 3×3+1], [Conv 16 4×4+2, Conv 16 3×3+1 Conv 16 3×3+1], FC 256